

```
where active = 1;
```

Table: *users*

The *users* table holds information about users in your mail system - for every domain:

```
create table users (
id bigserial primary key,
domain_id integer not null references domains (id) on update
restrict on delete cascade,
name character varying (50) not null,
password character (106) not null,
active boolean not null default true,
quota_kbytes integer not null default 0,
unique(name, domain_id)
);
```

The password field holds the *SHA512-CRYPT* hash produced by Dovecot as we'll see later when configuring Dovecot. The hash is exactly 106 characters long.

The *quota_kbytes* is used for deciding the amount of IMAP storage the user gets in kilobytes.

We need two views for the *users* table, the actual purpose of which will be understood when we configure Dovecot.

View: *active_users_passwd*

```
SELECT 999 AS userdb_uid, 992 AS userdb_gid,
'/var/vmail/' || d.name || '/' || u.name::text AS userdb_
home,
''::storage= ' || u.quota_kbytes AS userdb_quota_rule,
u.name || '@' || d.name AS "user",
u.password
FROM users u
JOIN domains d ON u.domain_id = d.id AND u.active = true
AND d.active = true;
```

999 is the ID of the user that will store all virtual domains data. 992 is the ID of the group to which users storing all data belong.

That will change depending on your distribution, so before creating this you need to create your *vmail* user and check for the *uid* and *gid*.

The quota rule will be explained in Dovecot configuration.

View: *active_users_userdb*

```
SELECT 999 AS uid, 992 AS gid,
'/var/vmail/' || d.name || '/' || u.name AS home,
u.name || '@' || d.name AS "user"
FROM users u
JOIN domains d ON u.domain_id = d.id AND u.active = true
AND d.active = true;
```

Table: *user_aliases*

This table is used for storing aliases to user accounts:

```
create table user_aliases (
id bigserial primary key,
domain_id integer not null references domains(id) on update
restrict on delete cascade,
name character varying(50) not null,
destination character varying(50) not null,
active boolean not null default true,
unique(name, domain_id),
foreign key(destination, domain_id) references users(name,
domain_id) on update restrict on delete cascade);
```

We have two triggers here to avoid conflicting *alias* and *usernames*, and to check that a user exists when an alias is created. In PostgreSQL, the procedures that must be executed by a trigger are created first:

```
CREATE OR REPLACE FUNCTION check_duplicate_user_for_alias_
name()
RETURNS trigger
LANGUAGE plpgsql
AS $function$begin
```

```
perform 1 from users where name = NEW.name and domain_id =
NEW.domain_id;
```

```
if found then
raise exception 'User with name % already exists for domain %
and therefore cannot create alias', NEW.name, NEW.domain_id;
end if;
```

```
return NEW;
```

```
end;
$function$
```

```
CREATE OR REPLACE FUNCTION check_user_existence_for_user_
alias()
RETURNS trigger
LANGUAGE plpgsql
AS $function$begin
```

```
perform 1 from users where name = NEW.destination
and domain_id = NEW.domain_id;
```

```
if not found then
raise exception 'User % not found for creating alias %',
NEW.destination, NEW.name;
end if;
```

```
return NEW;
```